

LEARNING MODULES

- What is Automation?
- Manual vs Automation
- Why do we need Automation?
- Where Playwright fits in real projects?
- Why Playwright over Selenium & Cypress?
- JavaScript Basics for Automation
 - ✓ Print Values
 - ✓ Single line & Multi lines comment
 - ✓ Variables
 - ✓ Camel case & Pascal Case
 - ✓ Data Types
 - ✓ Operators
 - ✓ Conditional Statement

◆ What is Automation?

Automation means using tools or scripts to perform tasks automatically without human intervention.

☞ In software testing:

Instead of manually clicking buttons and checking results,
We write code/scripts that test the application
automatically

✓ Example:

Logging into a website → automated script does it in
seconds

◆ Manual vs Automation Testing

□ 🖥️ Manual Testing

Tester manually interacts with the application

No code required

Time-consuming and repetitive

Pros:

- Good for exploratory testing
- Easy to start

Cons:

- Slow
- Prone to human errors
- Not scalable

◆ Manual vs Automation Testing

□ Automation Testing

Uses tools + scripts to test automatically
Requires programming knowledge

Pros:

- Fast and repeatable
- High accuracy
- Saves time in long run

Cons:

- Initial setup takes time
- Needs coding skills

◆ Manual vs Automation Testing

Feature	Manual Testing	Automation Testing
Speed	Slow	Fast
Accuracy	Medium	High
Reusability	No	Yes
Effort	High	Low (after setup)

◆ Why do we need Automation?

Automation Testing is important:

- ☞ Applications are growing fast and complex
- ☞ Frequent releases (Agile / CI-CD)
- ☞ Repetitive test cases

Key Reasons :

- ☐☐ Saves time
- ↻ Handles repetitive tasks
- 📈 Improves test coverage
- 🚀 Supports continuous integration
- ☐ Runs tests overnight / anytime

◆ Where Playwright fits in real projects

Playwright is used in modern web application testing:

1. In real projects:
2. UI Testing (buttons, forms, flows)
3. End-to-End Testing (complete user journey)
4. API Testing (optional)
5. Cross-browser testing (Chrome, Firefox, Safari)

👉 **Example:**

User login → Add to cart → Checkout → Payment

Playwright automates the entire flow

◆ Why Playwright over Selenium & Cypress

🚀 Playwright Advantages:

1. Multi-browser support (out of the box) - Chromium, Firefox, WebKit
2. Faster execution - Built for modern browsers
3. Auto-waiting - No need for manual waits (less flaky tests)
4. Better handling of modern apps - Works well with React, Angular, dynamic content
5. Multiple tabs & contexts - Easy to handle multiple pages



Feature	Selenium	Cypress	Playwright
Language	Many	JS only	JS, TS, Python, C#, Java
Speed	Medium	Fast	Very Fast
Auto-wait	No	Yes	Advanced
Multi-tab	Difficult	No	Easy

◆ JavaScript Basics for Automation

How to Print the Values?

```
Console.log()
```

Single Line Comment:

```
// you can comment this line
```

Multi Line Comment:

```
/* Hi  
How are you!  
*/
```

Variables:

A variable in JavaScript is a container used to store data values.

It allows us to save, update, and reuse values in a program

◆ JavaScript Basics for Automation

Types of Variables

- `var` ✗ (old - avoid)

`var` is the old way of declaring variables in JavaScript (before `let` and `const`).

- `let` ✓ (most common)

`let` is used when the value can change during program execution.

- `const` ✓ (for fixed values)

`const` is used when the value should NOT change once assigned.

◆ JavaScript Basics for Automation

Var Examples:

Code :

```
var count = 1;  
count = 2; // allowed  
var count = 3; // also allowed  
console.log(count)
```

Output - 3

Why var is NOT recommended ✖?

```
var x = 10;  
var x = 20; // no error 😞
```

◆ JavaScript Basics for Automation

let Examples:

```
let status= "Pending"  
status = "Passed"; // value changed  
Let status= "failed" // not allowed error
```

const Examples:

```
Const status= "pending"  
status= "passed" // not allowed error
```

Camel Case:

First word starts lowercase, next words start uppercase (e.g., `userName`, `totalAmount`).

Pascal Case:

Every word starts with uppercase (e.g., `UserName`, `TotalAmount`).

◆ JavaScript Basics for Automation

Data Types:

Primitive Data types:

- String
- Number
- Boolean
- Null
- Undefined

Reference Data types

- Array
- Object
- Function

◆ JavaScript Basics for Automation

Primitive Data types:

String:

```
Const tool = "playwright";
```

A string is used to store text or characters and is always written inside quotes (" " or ' ')

number:

```
Const age = 5;
```

A number is used to store numeric values, including both integers and decimals.

◆ JavaScript Basics for Automation

Primitive Data types:

boolean:

```
Const popular = true;
```

```
Const popular = false;
```

Boolean stores only two values: true or false, mainly used for conditions and decision-making.

◆ JavaScript Basics for Automation

Primitive Data types:

`null:`

```
Const firstname = "Aravind";
```

```
Const middlename = null;
```

```
Const lastname = "purusothaman";
```

`null` represents an intentional absence of a value, meaning the variable exists but currently has no value.

◆ JavaScript Basics for Automation

Primitive Data types:

undefined:

```
let value;
```

undefined means a variable is declared but not assigned any value yet

◆ JavaScript Basics for Automation

Non - Primitive Data types or Reference Data Types:

Array:

```
const tools = ["Playwright", "Selenium", "Cypress"];
```

An array is used to store multiple values in a single variable, accessed using index positions.

Object

```
const user = { name: "Aravind", role: "QA" };
```

An object stores data in key-value pairs, making it easy to represent real-world entities.

◆ JavaScript Basics for Automation

Non - Primitive Data types or Reference Data Types:

function:

```
function login () {  
    console.log("Login successful");  
}
```

Login()

A function is a reusable block of code that performs a specific task when called.

◆ JavaScript Basics for Automation

Arithmetic Operators

- Addition
- Subtraction
- Multiply
- Division
- Modulus
- Increment
- Decrement

Comparison Operators

- Equal
- Strict Equal
- Not Equal
- Strict Not Equal
- Greater Than
- Less Than
- Greater Than or Equal
- Less Than or Equal

Logical Operators

- AND
- OR
- NOT

◆ JavaScript Basics for Automation

ARITHMETIC OPERATORS

Addition (+)

- Used to add two or more numeric values.
- It can also join (concatenate) strings
- `console.log(10 + 5); // 15`
- `console.log("Hello " + "JS"); // Hello JS`

Subtraction (-)

- Used to subtract one value from another.
- The result is always a number.
- `console.log(10 - 4); // 6`

◆ JavaScript Basics for Automation

ARITHMETIC OPERATORS

Multiplication (*)

- Used to multiply two or more values.
- Commonly used in calculations and formulas.
- `console.log(5 * 3); // 15`

Division (/)

- Used to divide one value by another.
- The result can be a decimal value.
- `console.log(10 / 3); // 3.3333`

◆ JavaScript Basics for Automation

ARITHMETIC OPERATORS

Modulus (%)

- Returns the remainder after division.
- Mostly used to check even/odd numbers.
- `console.log(10 % 3); // 1`

Increment (++)

- Increases a variable value by 1.
- Used in loops and counters.
- `let count = 5; Count++;`
`console.log(count); // 6`

◆ JavaScript Basics for Automation

ARITHMETIC OPERATORS

Decrement (--)

- Decreases a variable value by 1.
- Used in loops and countdown logic.
- **let count = 5; Count--; console.log(count); // 4**

◆ JavaScript Basics for Automation

COMPARISON OPERATORS

Equal (==)

- Compares only the value, not the datatype.
- Datatype conversion happens automatically.
- `console.log(5 == "5"); // true`

Strict Equal (===)

- Compares both value and datatype.
- Most recommended operator in real projects.
- `console.log(5 === "5"); // false`

◆ JavaScript Basics for Automation

COMPARISON OPERATORS

Not Equal (!=)

- Checks whether two values are not equal.
- Datatype is not considered.
- `console.log(5 != "5"); // false`

Strict Not Equal (!==)

- Checks value and datatype are not equal.
- Safer option compared to !=
- `console.log(5 !== "5"); // true`

◆ JavaScript Basics for Automation

COMPARISON OPERATORS

Greater Than (>)

- Checks if left value is greater than right value.
- Returns true or false.
- `console.log(10 > 5); // true`

Less Than (<)

- Checks if left value is less than right value.
- Returns true or false.
- `console.log(5 < 10); // true`

◆ JavaScript Basics for Automation

COMPARISON OPERATORS

Greater Than or Equal ($\geq$)

- Checks if value is greater than or equal to another.
- Equality is also considered.
- `console.log(18  $\geq$  18); // true`

Less Than or Equal ($\leq$)

- Checks if value is less than or equal to another.
- Equality is also considered.
- `console.log(10  $\leq$  8); // false`

◆ JavaScript Basics for Automation

LOGICAL OPERATORS

AND (&&)

- Returns true only if all conditions are true.
- Used when multiple conditions must pass.
- `console.log(true && false); // false`

OR (||)

- Returns true if any one condition is true.
- Used when at least one condition is enough.
- `console.log(true || false); // true`

◆ JavaScript Basics for Automation

LOGICAL OPERATORS

NOT (!)

- Reverses the boolean value.
- True becomes false and false becomes true.
- `console.log(!true); // false`

◆ JavaScript Basics for Automation

CONDITIONAL STATEMENT

if statement

- Used to execute code only when a condition is true.
- Condition must return true or false.

```
let age = 20;  
  
if (age >= 18) {  
    console.log("Eligible to vote");  
}
```

◆ JavaScript Basics for Automation

CONDITIONAL STATEMENT

if-else statement

- Executes one block if condition is true.
- Executes else block when condition is false

```
let marks = 35;
```

```
if (marks >= 40) {  
    console.log("Pass");  
}  
else {  
    console.log("Fail");  
}
```

◆ JavaScript Basics for Automation

CONDITIONAL STATEMENT

else-if statement

- Used when multiple conditions are involved.
- Conditions are checked top to bottom.

```
let score = 85;

if (score >= 90) {
    console.log("Grade A");
}
else if (score >= 75) {
    console.log("Grade B");
}
else if (score >= 50) {
    console.log("Grade C");
}
else {
    console.log("Fail");
}
```

◆ JavaScript Basics for Automation

CONDITIONAL STATEMENT

Switch-Case

- Used when you compare one value with many possible values.
- More readable than multiple else if.

```
let day = 3;
switch (day) {
  case 1:
    console.log("Monday");
    break;
  case 2:
    console.log("Tuesday");
    break;
  case 3:
    console.log("Wednesday");
    break;
  default:
    console.log("Invalid day");
}
```